Unit – 7 File System Interface Management

File

A file is a named collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tapes and optical disks. In general, a file is a sequence of bits, bytes, lines or records whose meaning is defined by the files creator and user.

A file system stores and organizes data and can be thought of as a type of index for all the data contained in a storage device. Different OS have different file systems. The common file system used in Windows are FAT and NTFS.

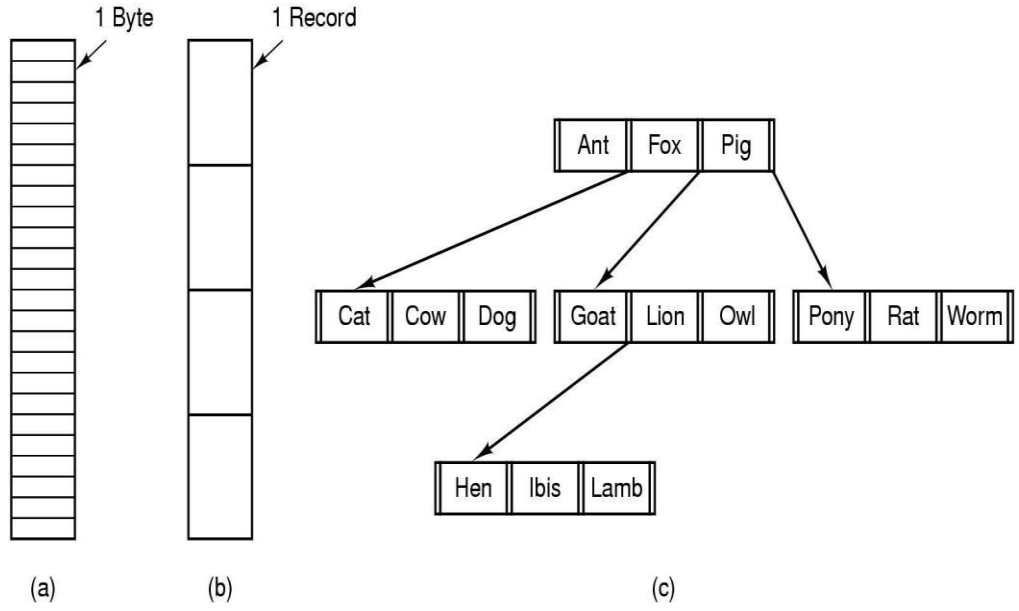
File Naming

When a process creates a file, it gives the file name; while process terminates, the file continue to exist and can be accesses by other processes.
A file is named, for the convenience of its human users, and it is referred to by its name. A name is string of characters. The string may be of digits or special characters (eg. 2, !,% etc). Some system differentiate between the upper and lower case character, whereas other system consider the equivalent (like Unix and MS-DOS). Normally the string of maximum 8 characters are legal file name (e.g., in DOS), but many recent system support as long as 255 characters (eg. Windows 2000 or above).

Many OS support two-part file names; separated by period; the part following the period is called the file extension and usually indicates something about the file (e.g., file.c – C source file). But in some system it may have two or more extension such as in Unix proc.c.Z - C source file compressed using Ziv-Lampel algorithm. In some system (e.g., Unix), file extension are just conventions; in other system it requires (e.g., C compiler must requires .c source file).

File Structure

Files must have structure that is understood by OS. Files can be structured in several ways. The most common structures are: Unstructured, Record Structured, Tree Structured.



a) Unstructured b) Record structured c) Tree structured

Unstructured:

Consist of unstructured sequence of bytes or words. OS does not know or care what is in the file. Any meaning must be imposed by user level programs. It provides maximum flexibility; user can put anything they want and name them anyway that is convenient. Both Unix and Windows use these approach.

Record Structured:

A file is a sequence of fixed-length records, each with some internal structure. Each read operation returns one records, and write operation overwrites or append one record. Many old mainframe systems use this structure.

Tree Structured:

File consists of tree of records, not necessarily all the same length. Each containing a key field in a fixed position in the record, sorted on the key to allow the rapid searching. The operation is to get the record with the specific key. Used in large mainframe for commercial data processing.

File Types

Many OS supports several types of files

Regular files: contains user information, are generally ASCII or binary.

Directories: system files for maintaining the structure of file system.

Character Special files: related to I/O and used to model serial I/O devices such as terminals, printers, and networks.

Block special files: used to model disks.

ASCII files: Consists of line of text. Each line is terminated either by carriage return character or by line feed character or both. They can be displayed and printed as is and can be edited with ordinary text editor.

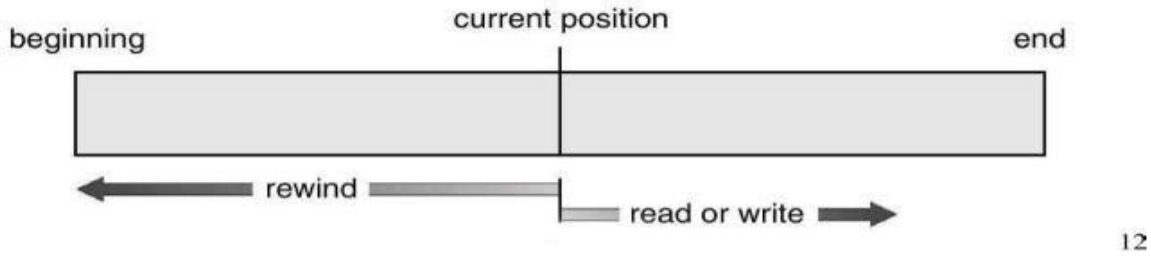
Binary files: Consists of sequence of byte only. They have some internal structure known to programs that use them (e.g., executable or archive files). Many OS use extension to identify the file types; but Unix like OS use a magic number to identify file types.

File Access

It defines the way in which the contents of the file are accessed. It can be sequential and direct access.

Sequential access:

It is the simplest access method. Information in the file is accessed in order, one record after another. It is convenient when the storage medium is the magnetic tape. It was used in many early systems.



12

Advantages:

- Simple and easier to understand.
- Easier to organize and maintain.
- Loading and reading a record requires only the Record Key.
- It is efficient and economical if the number of file records to be processed is high.
- Relatively inexpensive input/output media and devices may be used.
- Errors in the files remains localized.

Disadvantages:

- It doesn't support modern technologies that require fast access to stored records.

Direct Access:

Files whose bytes or records can be read in any order. Based on disk model of file, since disks allow random access to any block. It is used for immediate access to large amounts of information. When a query concerning a particular subject arrives, we compute which block contain the answer, and then read that block directly to provide desired information.

Advantages:

- Fast retrieval of records.
- The records can be different size.

Disadvantages:

- Data may be accidentally erased or overwritten unless special precautions are taken.
- Less efficient in the use of storage space.
- Expensive hardware and software are required.
- Relatively complex and costly to design and develop.

File Attributes

In addition to name and data, all other information about file is termed as file attributes. The file attributes may vary from system to system. Some common attributes are listed here.

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file has last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

File Operations

OS provides system calls to perform operations on files. Some common calls are:

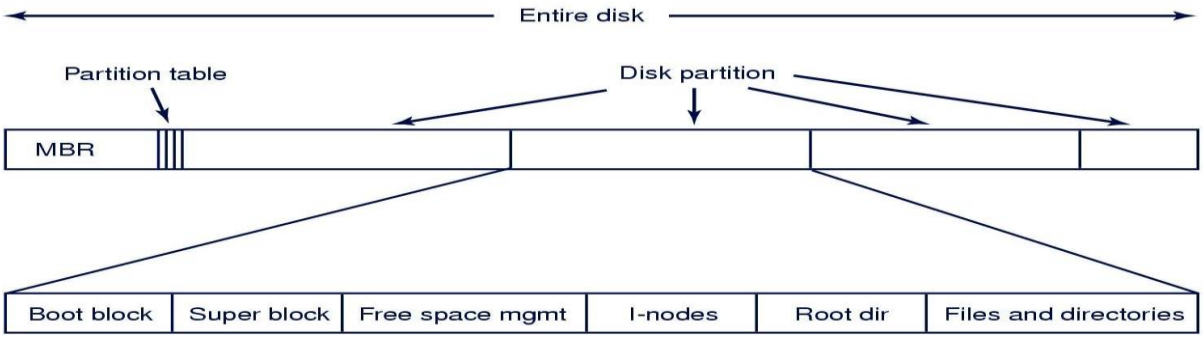
File operations	Explanation
Create	If disk space is available it create new file without data.
Delete	Deletes files to free up disk space.
Open	Before using a file, a process must open it.
Close	When access is completed, the file should be closed to free up the internal table space.
Read	Reads data from file.
Append	Adds data at the end of the file.
Seek	Repositions the file pointer to a specific place in the file.
Get attributes	Returns file attributes for processing.
Set attributes	To set the user settable attributes when file changed.
Rename	Rename file.

File-System Implementation

File-System Layout

A file system is a set of files, directories and other structures. It maintains information and identify where a file or directory's data is located on the disk.

Disks are divided up into one or more partitions, with independent file system on each partition.



MBR (Master Boot Record) – used to boot the computer.

Partition Table – gives the starting and ending addresses of each partition.

Boot block – the program in boot block loads the operating system contained in that partition.

Super block – contains all the key parameters about the file system and is read into memory when the computer is booted.

Allocation Methods

The allocation method defines how the files are stored in the disk blocks. The common allocation methods are: Contiguous allocation, linked allocation, and Indexed allocation.

Contiguous Allocation: Each file occupy a set of contiguous block on the disk. Disk addresses define a linear ordering on the disk. File is defined by the disk address and length in block units. With 2-KB blocks, a 50-KB file would be allocated 25 consecutive blocks.

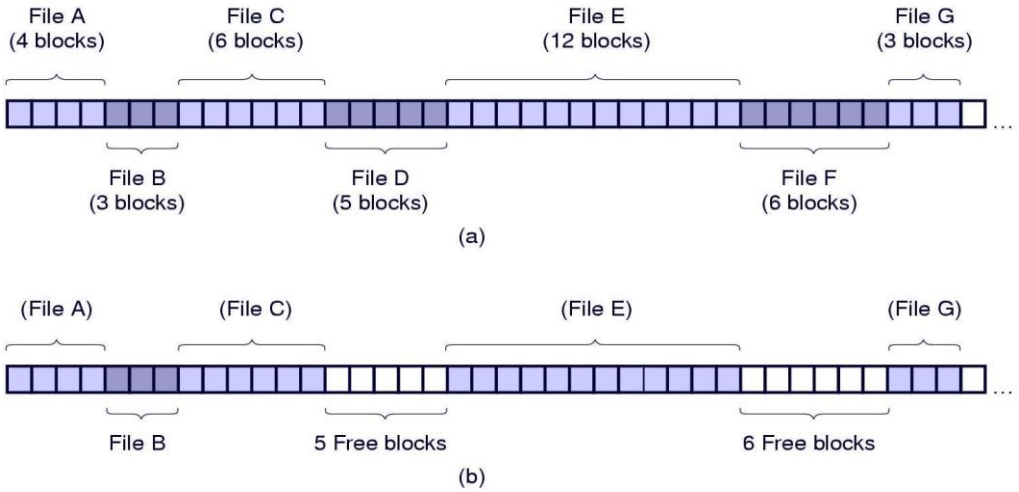
Both sequential and direct access can be supported by contiguous allocation.

Advantages:

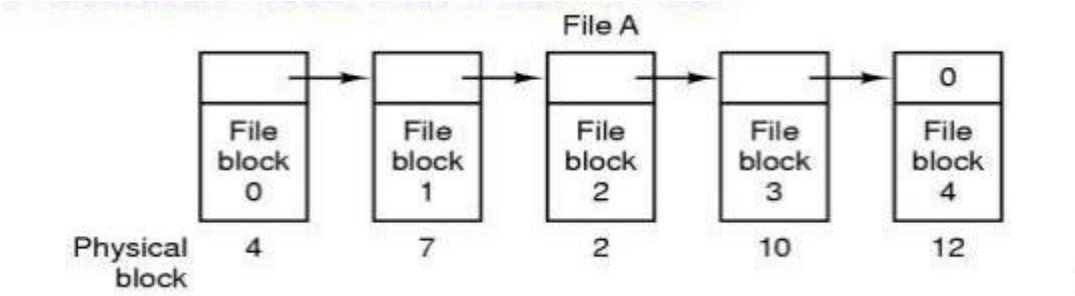
- Simple to implement; accessing a file that has been allocated contiguously is easy.
- High performance; the entire file can be read in single operation i.e. decrease the seek time.

Problems:

- Fragmentation: when files are allocated and deleted the free disk space is broken into holes.
- Dynamic-storage-allocation problem: searching of right holes. Required pre-information of file size.



Linked Allocation: Each file is a linked list of disk blocks; the disk block may be scattered anywhere on the disk. Each block contain the pointer to the next block of same file. To create the new file, we simply create a new entry in the directory; with linked allocation, each directory entry has a pointer to the first disk block of the file.



Problems:

- It solves all problems of contiguous allocation but it can used only for sequential access file; random access is extremely slow.
- Each block access requires disk seek. It also requires space for pointer.

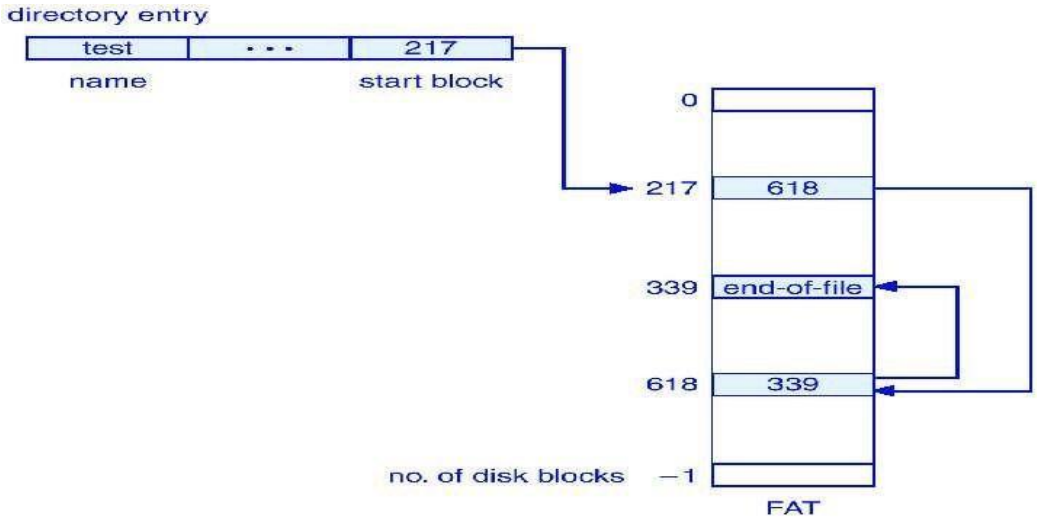
Solution: Using File Allocation Table (FAT).

Linked Allocation using FAT: The FAT is used as in a linked list. The directory entry contains the block number of the first block of the file. The FAT is looked to find next block until a special end-of-file value is reached.

Advantages:

- The entire block is available for data.
- Results the significant number of disk seek; random access time is improved.

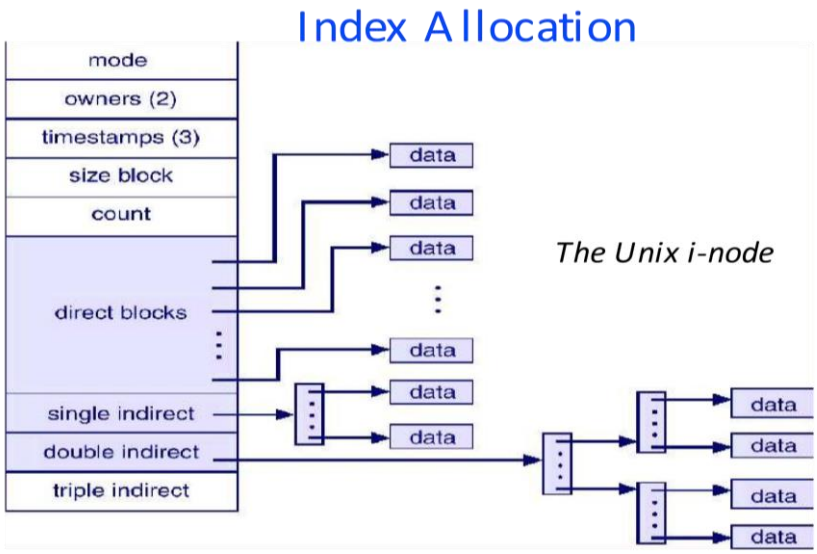
Problems: The entire table must be in memory all the time to make it work. With 20GB disk and 1KB block size, the table needs 20 millions entries. Suppose each entry requires 3 bytes. Thus the table will take up 60MB of main memory all the time.



Index Allocation: To keep the track of which blocks belongs to which file, each file has data structure (i-node) that list the attributes and disk address of the disk block associate with the file. Each i-node are stored in a disk block, if a disk block is not sufficient to hold i-node it can be multileveled. Independent to disk size. If i-node occupies n bytes for each file and k files are opened, the total memory by i-nodes is kn bytes.

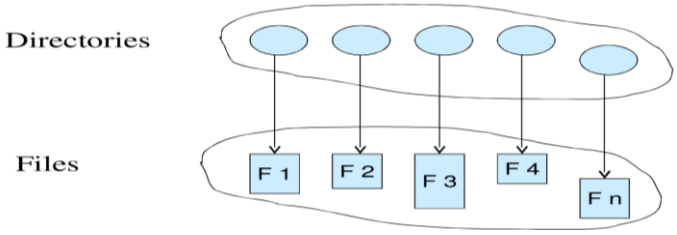
Advantages: Far smaller than space occupied by FAT.

Problem: This can suffer from performance issue.



Directories

A directory is a node containing information about files and sub-directories.



Directory structures

Directories can have different structures:

Single-Level-Directory:

All files are contained in the same directory. It is easy to support and understand; but difficult to manage large amount of files and to manage different users.

Two-Level-Directory:

Separate directory for each user. It is used on a multiuser computer and on a simple network computers. It has problem when users want to cooperate on some task and to access one another's files. It also cause problem when a single user has large number of files.

Hierarchical-Directory:

Generalization of two-level-structure to a tree of arbitrary height. This allow the user to create their own subdirectories and to organize their files accordingly. This allow to share the directory for different user acyclic-graph-structure is used. Nearly all modern file systems are organized in this manner.

Path Names

Absolute and Relative path names.

Absolute Path Name:

Path name starting from root directory to the file.

e.g. In Unix: /usr/user1/bin/lab2.

Path separated by / in Unix and \ in windows.

Relative Path Name:

Concept of working directory. A user can designate one directory as the current working directory, in which case all path names not beginning at the root directory are taken relative to the working directory. E.g., bin/lab2 is enough to locate same file if current working directory is /usr/user1.

Directory Operations

Some of the key directory operations:

Operation	Explanation
Create	A directory is created.
Delete	A directory is deleted. Only an empty directory is deleted.
Opendir	Directory can be read. For e.g. to list all files in a directory, a listing program opens the directory to read out the names of all the files it contain.
Closedir	When a directory has been read, it should be closed to free up internal table space.
Rename	Rename the directory same as a file.
Link	It is a technique that allow a file to appear in more than one directory.
Unlink	A directory entry is removed.

Directory Implementation

The directory entry provides the information needed to find the disk block of the file. The file attributes are stored in the directory.

Linear List: It uses the list of the file names with pointer to the data blocks. It requires linear search to find a particular entry.

Advantages: Simple to implement.

Problem: linear search to find the file.

Hash Table: It consist linear list with hash table. The hash table takes a value computed from the file name and returns a pointer to the file name in the linear list. If that key is already in use, a linked list is constructed.

Advantages: Decreases the file search time.

Problem: It greatly fixed size and dependence of the hash function on that size.

Block Size

Nearly all file systems chop files up into fixed-size block. Using a large size result the wastage of disk space. Using a small size increases seek and the rotational delay- low data access rate.

Free-Space Management

To keep track of free blocks, system maintains the frees-pace list. The free-space-list records all free blocks- those not allocated to some file or directory. To create a file, system search the free-space-list for required amount of space, and allocate that space to the new file and removed from free-space-list. When a file is deleted, its disk space is added to the free-space-list.

The free-space-list can be implemented in two ways: Bitmap, Linked list.

Bitmap: Each block is represented by a bit. If the block is free, the bit is 1; if the block is allocated the bit is 0. A disk with n blocks requires a bitmap with n bits.

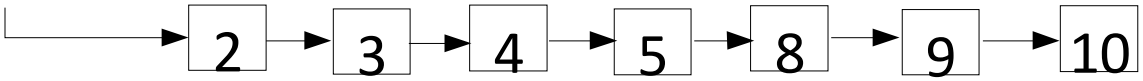
Ex: Consider a disk where blocks 2,3,4,5,8,9,10,11,12,13,17,18,..... are free, and rest are allocated. The free space bit map would be 011110011111100010.....

Advantages: Simple and efficient in finding first free block, or n consecutive free blocks.

Problems: Inefficient unless the entire bitmap is kept in main memory. Keeping in main memory is possible only for small disk, when disk is large the bitmap would be large. Many system use bitmap method.

Linked List: Link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory. The first block contains a pointer to the next free block.

The previous example can be represented in linked as



Advantages: Only one block is kept in memory.

Problems: Not efficient; to traverse list, it must read each block.

Protection

Protection refers to prevent malicious use of the system by users or programs. It ensures that each shared resource is used only in accordance with system policies, which may be set either by system designers or by system administrators.

Access control and authentication are the important tools used for protection.

Access control is a process that allows users to grant access and certain privileges to systems, resources, or information.

Access control list and access control matrix are two terms associated with the access control process.

Access Control List

Access Control List (ACL) refers to the permissions attached to an object that specifies which users are granted access to that object. Furthermore, it also specifies the operations the users can perform using that object.

A file system ACL contains entries that specify individual user or group rights to specific system objects such as programs, processes, and files. These entries are called access control entries (ACEs) in the Microsoft Windows NT, OpenVMS, UNIX, and Mac OS X operating systems. Moreover, each system object has a security attribute to recognize its ACL.

Access Control Matrix

Access control Matrix allows implementing protection model. This matrix contains rows and columns. Rows represent the domain. It can be a user, process or a procedure domain. Columns, on the other hand, represent the objects or resources. An expel Access Control Matrix is as follows.

Domain/ Object	File 1	File 2	File 3	Printer
D1	Read		Read	
D2				Print
D3			Execute	
D4		Write		

Each entry in the matrix represents access right information. In the entry access (Di, Oj), Di represents a process in the domain while Oj represents an object or the resource. According to the above matrix, a process in domain 1 can read File 1. A process in domain 2 can take printouts, and a process in domain 3 can execute File 3. Moreover, a process in domain 4 can write to File 2.

Access Control List (ACL)	Access Control Matrix
Access control list is a list of permissions attached to an object in a computer file system, database or a network.	Access control matrix is an abstract, formal security model for protection state in computer systems that characterize the rights of each subject with respect to every object in the system.
Access control list defines the access rights each user has to a particular system object such as a file directory or individual files.	Access control matrix defines a subject's access rights such as read, write, and execute on an object.